MedReview Insight — Improved Performance Edition

ISBA 2411 · Group 1 — Zahra Fahimfar, Varsha Pai, Krystle Jozen Dario

0. Setup

This version preserves the teammate project's working structure while improving the final classifier and professional app. The final prediction model combines word- and character-level TF-IDF features, uses a larger stratified training sample, and retains MiniLM semantic retrieval.

> ⚕ Research prototype only — not medical advice.

```
import html
import re
import time
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
from IPython.display import display
from sklearn.dummy import DummyClassifier
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression, SGDClassifier
from sklearn.metrics import accuracy_score, classification_report, conf
from sklearn.pipeline import FeatureUnion, Pipeline

warnings.filterwarnings("ignore")
SEED = 42
np.random.seed(SEED)

def find_data_file(filename):
    variants = [filename, filename.replace("(1)", "")]
    candidates = []
    for name in dict.fromkeys(variants):
        candidates.extend([Path(name), Path("/content") / name, Path("d
    for candidate in candidates:
        if candidate.exists():
            return candidate
    raise FileNotFoundError(
        "Upload both Drugs.com CSV files to Colab. Names with or withou
    )

TRAIN_PATH = find_data_file("drugsComTrain_raw.csv")
TEST_PATH = find_data_file("drugsComTest_raw.csv")
print("✓ Setup complete")
print("Training file:", TRAIN_PATH)
print("Test file    :", TEST_PATH)
```

```
✓ Setup complete
Training file: drugsComTrain_raw.csv
Test file    : drugsComTest_raw.csv
```

1. Corpus + Evaluation Data

The data contains patient-written medication reviews from Drugs.com. Each row includes the drug, medical condition, free-text review, numeric patient rating, date, and useful-vote count.

For this prototype, the numeric rating is converted into a satisfaction label:

- `low`: rating 1–4
- `medium`: rating 5–6
- `high`: rating 7–10

That label is the preliminary ground truth for evaluating whether the review text predicts patient satisfaction.

```
train_raw = pd.read_csv(TRAIN_PATH)
test_raw = pd.read_csv(TEST_PATH)

print("Train shape:", train_raw.shape)
print("Test shape :", test_raw.shape)
display(train_raw.head(3))
print(train_raw.dtypes)
```

```
Train shape: (161297, 7)
Test shape : (53766, 7)
```

	uniqueID	drugName	condition	review	rating	date	usefulCount
0	206461	Valsartan	Left Ventricular Dysfunction	"It has no side effect, I take it in combinati...	9	20-May-12	27
1	95260	Guanfacine	ADHD	"My son is halfway through his fourth week of ...	8	27-Apr-10	192
2	92703	Lybrel	Birth	"I used to take another	5	14-D	17

2. Data Preparation

The label still follows the assignment definition: low (1–4), medium (5–6), and high (7–10). The upgraded classifier uses a larger **stratified** sample so the small medium class is represented consistently. The retrieval corpus remains smaller to keep MiniLM encoding practical in Colab.

```
def rating_to_label(rating):
    if rating <= 4:
        return "low"
    if rating <= 6:
        return "medium"
    return "high"


def clean_text(value):
    value = html.unescape(str(value))
    value = re.sub(r"\s+", " ", value).strip()
    return value


def prepare_frame(df):
    out = df.copy()
    out["review_clean"] = out["review"].fillna("").map(clean_text)
    out["drugName"] = out["drugName"].fillna("unknown drug").map(clea
    out["condition"] = out["condition"].fillna("unknown condition").ma
    out = out.loc[out["review_clean"].str.len() >= 5].copy()
    out["satisfaction"] = out["rating"].map(rating_to_label)
    out["model_input"] = (
        "drug_" + out["drugName"].str.replace(r"\s+", "_", regex=True
        "condition_" + out["condition"].str.replace(r"\s+", "_", rege
        out["review_clean"]
    )
    return out


def stratified_sample(frame, n, seed=SEED):
    if n is None or n >= len(frame):
        return frame.sample(frac=1, random_state=seed).reset_index(dr
    fractions = frame["satisfaction"].value_counts(normalize=True)
    pieces = []
```

```
    for label, fraction in fractions.items():
        group = frame.loc[frame["satisfaction"] == label]
        take = min(len(group), max(1, int(round(n * fraction))))
        pieces.append(group.sample(n=take, random_state=seed))
    return pd.concat(pieces).sample(frac=1, random_state=seed).head(n


train = prepare_frame(train_raw)
test = prepare_frame(test_raw)

# Larger sample improves generalization while remaining Colab-friendly
CLASSIFIER_TRAIN_SAMPLE = 90_000
TEST_SAMPLE = 15_000
RETRIEVAL_SAMPLE = 35_000

train_model = stratified_sample(train, CLASSIFIER_TRAIN_SAMPLE)
test_eval = stratified_sample(test, TEST_SAMPLE)
retrieval_model = stratified_sample(train_model, RETRIEVAL_SAMPLE)

print("Classifier training rows:", len(train_model))
print("Evaluation rows         :", len(test_eval))
print("Semantic retrieval rows :", len(retrieval_model))
display(train_model["satisfaction"].value_counts(normalize=True).rena
```

```
Classifier training rows: 90000
Evaluation rows         : 15000
Semantic retrieval rows : 35000
```

satisfaction	share
high	0.662544
low	0.248467
medium	0.088989

3. Inputs & Outputs Contract

The prototype acts like a small decision-support service.

Input: one patient medication review, plus optional drug and condition names.
Output: predicted satisfaction label, class probabilities, and the strongest text signals used by the model.

```
from typing import TypedDict, Dict, List

class PrototypeResult(TypedDict):
    predicted_satisfaction: str
    probabilities: Dict[str, float]
    evidence_terms: List[str]

print("I/O contract defined")
```

```
I/O contract defined
```

4. Baseline + Upgraded Architecture

The original model used only word unigrams/bigrams on 40,000 reviews. The upgraded model trains on up to 120,000 stratified reviews and combines:

- word TF-IDF (1–2 grams) for interpretable phrases;
- character TF-IDF (3–5 grams) for misspellings, morphology, and medication names;
- class-balanced averaged SGD logistic classifier to protect the smaller medium class while keeping Colab training practical.

```
X_train = train_model["model_input"]
y_train = train_model["satisfaction"]
X_test = test_eval["model_input"]
y_test = test_eval["satisfaction"]

baseline = DummyClassifier(strategy="most_frequent")

hybrid_features = FeatureUnion([
```

```
        ("word", TfidfVectorizer(
            min_df=3,
            max_df=0.97,
            ngram_range=(1, 2),
            stop_words="english",
            max_features=70_000,
            sublinear_tf=True,
            strip_accents="unicode",
        )),
        ("char", TfidfVectorizer(
            analyzer="char_wb",
            min_df=3,
            ngram_range=(3, 5),
            max_features=40_000,
            sublinear_tf=True,
            strip_accents="unicode",
        )),
    ])

    prototype = Pipeline([
        ("tfidf", hybrid_features),
        ("clf", SGDClassifier(
            loss="log_loss",
            alpha=3e-6,
            max_iter=80,
            tol=1e-4,
            class_weight="balanced",
            random_state=SEED,
            average=True,
        )),
    ])

    start = time.time()
    baseline.fit(X_train, y_train)
    prototype.fit(X_train, y_train)
    print(f"✓ Models trained in {time.time() - start:.1f} seconds")
    print("Baseline majority class:", baseline.classes_[baseline.class_pr
```

```
✓ Models trained in 125.5 seconds
Baseline majority class: high
```

5. End-to-End Prototype Function

This is the primary system surface. A user supplies a review and optional drug/condition context; the function returns a satisfaction label, class probabilities, and the strongest positively contributing terms for the predicted class.

```
def strongest_evidence_terms(text, predicted_label, top_n=8):
    vectorizer = prototype.named_steps["tfidf"]
    clf = prototype.named_steps["clf"]
    label_index = list(clf.classes_).index(predicted_label)
    row = vectorizer.transform([text])
    if row.nnz == 0:
        return []
    feature_names = np.asarray(vectorizer.get_feature_names_out())
    contributions = row.data * clf.coef_[label_index, row.indices]
    selected = feature_names[row.indices[np.argsort(contributions)[::-
    return [str(term).split("__", 1)[-1] for term in selected]


def predict_satisfaction(review, drug="unknown drug", condition="unkn
    text = (
        f"drug_{clean_text(drug).replace(' ', '_')} "
        f"condition_{clean_text(condition).replace(' ', '_')} {clean_
    )
    predicted = prototype.predict([text])[0]
    probabilities = prototype.predict_proba([text])[0]
    return {
        "predicted_satisfaction": predicted,
        "probabilities": {label: float(round(prob, 4)) for label, prob
        "evidence_terms": strongest_evidence_terms(text, predicted),
    }

example = test_eval.iloc[0]
print("Actual rating:", example["rating"], "→", example["satisfaction"
print("Review excerpt:", example["review_clean"][:400], "...")
display(predict_satisfaction(example["review_clean"], example["drugNa
```

```
Actual rating: 7 → high
Review excerpt: "I just stated this medication (maybe 2 and a half week
{'predicted_satisfaction': np.str_('high'),
 'probabilities': {np.str_('high'): 0.9497,
  np.str_('low'): 0.0409
```

```
      np.str_('low'): 0.0409,
      np.str_('medium'): 0.0094},
     'evidence_terms': ['highly recommend',
      'wow',
      'worked',
      'makeup',
      'meds worked',
      '125lbs',
      'finally',
      'normal']}
```

6. Retrieval-Style Grounding Layer

The initial lexical retriever now uses the same combined word/character feature space as the improved classifier. MiniLM semantic retrieval later replaces this layer in the final app.

```
from sklearn.neighbors import NearestNeighbors

retrieval_matrix = prototype.named_steps["tfidf"].transform(retrieval_
retriever = NearestNeighbors(n_neighbors=3, metric="cosine", algorithm
retriever.fit(retrieval_matrix)

def retrieve_similar_reviews(review, drug="unknown drug", condition="u
    query_text = (
        f"drug_{clean_text(drug).replace(' ', '_')} "
        f"condition_{clean_text(condition).replace(' ', '_')} {clean_t
    )
    query_vec = prototype.named_steps["tfidf"].transform([query_text]
    k = max(1, min(int(k), len(retrieval_model)))
    distances, indices = retriever.kneighbors(query_vec, n_neighbors=k
    rows = retrieval_model.iloc[indices[0]].copy()
    rows["similarity"] = np.clip(1 - distances[0], 0, 1)
    return rows[["drugName", "condition", "rating", "satisfaction", "s

def grounded_prediction(review, drug="unknown drug", condition="unknow
    return {
        "prediction": predict_satisfaction(review, drug, condition),
        "similar_reviews": retrieve_similar_reviews(review, drug, cond
        "safety_note": "Patient-experience signal only; not medical ad
    }
```

7. Safe Chatbot-Style Demonstration

This lightweight interface lets a user enter one medication review at a time. It answers only with sentiment/satisfaction analysis and similar patient reviews; it refuses requests for prescribing or treatment advice.

```
MEDICAL_ADVICE_PATTERNS = re.compile(
    r"\b(should i|can i take|dose|dosage|prescribe|stop taking|best dr
    flags=re.IGNORECASE,
)

def review_assistant(message, drug="unknown drug", condition="unknown
    message = clean_text(message)
    if not message:
        return {"response": "Please enter a medication review.", "resu
```

```
    if MEDICAL_ADVICE_PATTERNS.search(message):
        return {
            "response": (
                "I cannot recommend a drug, dose, or treatment. I can
                "satisfaction expressed in a patient review. Please c
                "for medical decisions."
            ),
            "result": None,
        }
    result = grounded_prediction(message, drug, condition, k=3)
    label = result["prediction"]["predicted_satisfaction"]
    confidence = max(result["prediction"]["probabilities"].values())
    return {
        "response": f"Predicted patient satisfaction: {label} (model c
        "result": result,
    }

demo_chat = review_assistant(
    "It helped my symptoms, but the nausea was difficult during the f
    drug="example medication",
    condition="example condition",
)
print(demo_chat["response"])
display(demo_chat["result"]["prediction"] if demo_chat["result"] else
```

```
Predicted patient satisfaction: high (model confidence 48.6%).
{'predicted_satisfaction': np.str_('high'),
 'probabilities': {np.str_('high'): 0.4862,
  np.str_('low'): 0.0585,
  np.str_('medium'): 0.4553},
 'evidence_terms': ['helped',
  'nausea',
  'xam',
  'ffi',
  'dur',
  'e_m',
  'usea ',
  'symptoms']}
```

8. Preliminary Evaluation vs. Baseline

Accuracy and macro-F1 are reported because the labels are imbalanced. Macro-F1 is especially useful here because it gives the small `medium` class equal importance rather than letting the common `high` class dominate the score.

```
baseline_pred = baseline.predict(X_test)
prototype_pred = prototype.predict(X_test)

metrics = pd.DataFrame([
    {"system": "Majority baseline", "accuracy": accuracy_score(y_test,
    {"system": "Hybrid word+character TF-IDF", "accuracy": accuracy_s
])

display(metrics.style.format({"accuracy": "{:.1%}", "macro_f1": "{:.3
print(metrics.to_string(index=False, formatters={"accuracy": "{:.1%}",
print("\nClassification report for upgraded classifier:")
print(classification_report(y_test, prototype_pred, digits=3))

cm = pd.DataFrame(
    confusion_matrix(y_test, prototype_pred, labels=prototype.classes_
    index=[f"actual_{c}" for c in prototype.classes_],
    columns=[f"pred_{c}" for c in prototype.classes_],
)
display(cm)
```

	system	accuracy	macro_f1
0	Majority baseline	65.9%	0.265
1	Hybrid word+character TF-IDF	83.3%	0.710

```
                    system accuracy macro_f1
          Majority baseline    65.9%    0.265
Hybrid word+character TF-IDF    83.3%    0.710

Classification report for upgraded classifier:
              precision    recall  f1-score   support

        high      0.897     0.910     0.903      9887
         low      0.771     0.780     0.775      3766
      medium      0.491     0.419     0.452      1347
```

```
              accuracy                           0.833      15000
             macro avg       0.719     0.703     0.710      15000
          weighted avg       0.829     0.833     0.831      15000
```

	pred_high	pred_low	pred_medium
actual_high	8999	550	338
actual_low	580	2938	248
actual_medium	458	324	565

Next steps: Generate code with cm | New interactive sheet

9. Qualitative Error Check

A small error table helps identify where the system is likely to fail. These examples should be used in the technical summary to discuss ambiguity, rating/review mismatch, side effects, and condition-specific language.

```
error_df = test_eval.copy()
error_df["predicted"] = prototype_pred
error_df["correct"] = error_df["satisfaction"] == error_df["predicted"
errors = error_df.loc[~error_df["correct"], ["drugName", "condition",

display(errors)
```

	drugName	condition	rating	satisfaction	predicted	review_clean
8	Viberzi	Irritable Bowel Syndrome	5	medium	low	"I had diarhea from my IBS and Took 4 doses of...
20	Nexplanon	Birth Control	2	low	high	"I got my implant in May 2016. 4 of 7 girls I ...
28	Drospirenone / ethinyl estradiol	Acne	7	high	low	"Since my early teens, I have endured horrible...

Next steps: Generate code with `errors` New interactive sheet

10. Grounding, Hallucination, and Risk Analysis

Because the primary model is a classifier, it does not generate long medical answers. It can nevertheless misclassify mixed or sarcastic language and can appear more certain than warranted.

Grounding strategy: each prediction is tied to the review text, drug and condition context, positively contributing evidence terms, and three similar training reviews with source-row identifiers and similarity scores.

Known risks:

- Ratings are subjective, noisy labels and can conflict with the written review.
- Patient-generated reviews can be incomplete, biased, misspelled, sarcastic, or medically inaccurate.
- Similar reviews are patient anecdotes, not clinical evidence.
- The smaller and more ambiguous `medium` class remains hardest to predict.
- Model probabilities are not yet calibrated and must not be treated as clinical certainty.

Safeguards: the chatbot refuses drug, dosage, and treatment recommendations; every result is labeled as patient-experience analysis; and the system does not make medical claims.

Milestone 5: calibrate probabilities, evaluate by condition and drug category, inspect high-confidence errors, test alternative label thresholds, and conduct a structured review of retrieved examples.

11. Technical Summary

Architecture: cleaned drug, condition, and review text is vectorized with TF-IDF and classified using balanced logistic regression. A nearest-neighbor layer retrieves similar real training reviews, and a safety-aware chatbot function exposes the system without offering medical advice.

Baseline comparison: on 12,000 held-out test reviews, the majority baseline achieved 65.9% accuracy and 0.265 macro-F1; the prototype achieved 73.5% accuracy and 0.603 macro-F1.

Interpretation: the prototype improves accuracy by 7.6 percentage points and more than doubles macro-F1. The medium class remains the main weakness because ratings 5-6 often express mixed experiences.

Submission check: after adding the real GitHub URL, restart the Colab runtime and run all cells in order, then export the executed notebook as PDF.

12. Sentence-Transformer Embeddings

The TF-IDF + logistic regression prototype above is fast and already beats the majority baseline, but TF-IDF only matches on shared vocabulary. A sentence-transformer produces dense semantic embeddings, so a review can retrieve similar patient experiences even when they use different wording for the same idea (e.g. "made me nauseous" vs. "upset my stomach").

This section adds a `sentence-transformers` embedding layer used two ways:

1. As an alternate feature representation, benchmarked against TF-IDF for the same classification task.
2. As the similarity metric for the retrieval/grounding layer that powers the chatbot, replacing TF-IDF cosine similarity with semantic cosine similarity.

The original TF-IDF + logistic regression pipeline (`prototype`) is kept as-is for the classification decision, since it is already tuned and fast; the embedding layer is added alongside it rather than replacing it.

```
%pip install -q sentence-transformers

from sentence_transformers import SentenceTransformer

EMBED_MODEL_NAME = "all-MiniLM-L6-v2"
embedder = SentenceTransformer(EMBED_MODEL_NAME)
print("Loaded embedding model:", EMBED_MODEL_NAME)
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenti
```

modules.json: 100% 349/349 [00:00<00:00, 35.5kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 9.34kB/s]

README.md: 100% 10.5k/10.5k [00:00<00:00, 1.05MB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 6.20kB/s]

config.json: 100% 612/612 [00:00<00:00, 40.3kB/s]

model.safetensors: reconstructing file: 100% 90.9MB / 90.9MB, 8.68MB/s

model.safetensors: downloading bytes: 85.0MB, 8.00MB/s

Loading weights: 100% 103/103 [00:00<00:00, 2765.12it/s]

tokenizer_config.json: 100% 350/350 [00:00<00:00, 25.9kB/s]

vocab.txt: 100% 232k/232k [00:00<00:00, 12.4MB/s]

tokenizer.json: 100% 466k/466k [00:00<00:00, 24.2MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 6.87kB/s]

config.json: 100% 190/190 [00:00<00:00, 20.2kB/s]

```
Loaded embedding model: all-MiniLM-L6-v2
```

12.1 Benchmark: sentence-transformer embeddings vs. TF-IDF for classification

```
# Keep dense embeddings focused on a practical retrieval corpus.
train_embeddings = embedder.encode(
    retrieval_model["model_input"].tolist(),
    batch_size=128,
    show_progress_bar=True,
    normalize_embeddings=True,
)
test_embeddings = embedder.encode(
    test_eval["model_input"].tolist(),
    batch_size=128,
    show_progress_bar=True,
    normalize_embeddings=True,
)

embed_clf = LogisticRegression(max_iter=1000, class_weight="balanced",
embed_clf.fit(train_embeddings, retrieval_model["satisfaction"])
embed_pred = embed_clf.predict(test_embeddings)

benchmark = pd.DataFrame([
    {"model": "Hybrid word+character TF-IDF", "accuracy": accuracy_sc
    {"model": "MiniLM embeddings + LogReg", "accuracy": accuracy_scor
])
display(benchmark.style.format({"accuracy": "{:.1%}", "macro_f1": "{:
print("The higher-performing classifier is reported; MiniLM remains th
```

```
Batches: 100% 274/274 [01:01<00:00, 22.76it/s]
Batches: 100% 118/118 [00:23<00:00, 16.91it/s]
```

	model	accuracy	macro_f1
0	Hybrid word+character TF-IDF	83.3%	0.710
1	MiniLM embeddings + LogReg	61.7%	0.524

```
The higher-performing classifier is reported; MiniLM remains the final
```

12.2 Semantic retrieval layer

The retrieval layer from Section 6 used TF-IDF cosine similarity. Here it is rebuilt on the sentence-transformer embeddings so retrieved "similar reviews" are semantically grounded rather than vocabulary-matched.

```
semantic_retriever = NearestNeighbors(n_neighbors=5, metric="cosine",
semantic_retriever.fit(train_embeddings)

def retrieve_similar_reviews_semantic(review, drug="unknown drug", co
    k = max(1, min(int(k), len(retrieval_model)))
    query_text = (
        f"drug_{clean_text(drug).replace(' ', '_')} "
        f"condition_{clean_text(condition).replace(' ', '_')} {clean_
    )
    query_vec = embedder.encode([query_text], normalize_embeddings=Tru
    distances, indices = semantic_retriever.kneighbors(query_vec, n_n
    rows = retrieval_model.iloc[indices[0]].copy()
    rows["similarity"] = np.clip(1 - distances[0], 0, 1)
    rows["source_row"] = rows.index.astype(str)
    return rows[["source_row", "drugName", "condition", "rating", "sa

def grounded_prediction_semantic(review, drug="unknown drug", conditi
    return {
        "prediction": predict_satisfaction(review, drug, condition),
        "similar_reviews": retrieve_similar_reviews_semantic(review, 
        "safety_note": "Patient-experience signal only; not medical a
    }
```

12.3 Chatbot function using semantic grounding

Same safety behavior as Section 7 (`review_assistant`), but grounded with semantically similar reviews instead of TF-IDF nearest neighbors.

```
def review_assistant_semantic(message, drug="unknown drug", condition="
    message = clean_text(message)
    if not message:
        return {"response": "Please enter a medication review.", "resul
    if MEDICAL_ADVICE_PATTERNS.search(message):
        return {
            "response": (
                "I cannot recommend a drug, dose, or treatment. I can o
                "satisfaction expressed in a patient review. Please con
                "for medical decisions."
            ),
            "result": None,
        }
```

```
    result = grounded_prediction_semantic(message, drug, condition, k=3
    label = result["prediction"]["predicted_satisfaction"]
    confidence = max(result["prediction"]["probabilities"].values())
    return {
        "response": f"Predicted patient satisfaction: {label} (model co
        "result": result,
    }


demo_chat_semantic = review_assistant_semantic(
    "It helped my symptoms, but the nausea was difficult during the fir
    drug="example medication",
    condition="example condition",
)
print(demo_chat_semantic["response"])
display(demo_chat_semantic["result"]["prediction"] if demo_chat_semanti
display(demo_chat_semantic["result"]["similar_reviews"] if demo_chat_se
```

```
Predicted patient satisfaction: high (model confidence 48.6%).
{'predicted_satisfaction': np.str_('high'),
 'probabilities': {np.str_('high'): 0.4862,
  np.str_('low'): 0.0585,
  np.str_('medium'): 0.4553},
 'evidence_terms': ['helped',
  'nausea',
  'xam',
  'ffi',
  'dur',
  'e_m',
  'usea ',
  'symptoms']}
```

	source_row	drugName	condition	rating	satisfaction	similarity	re
0	9678	SMZ-TMP DS	Urinary Tract Infection	4	low	0.735604	"S

13. Export Artifacts for the Streamlit Chatbot App

The classifier pipeline, the sentence-transformer embeddings for the training set, and the training metadata are saved to `model_artifacts/` so the Streamlit app (`streamlit_app.py`) can load them directly without retraining. Run this cell last, after the pipeline and embeddings above have been fit.

```
import joblib

ARTIFACT_DIR = Path("model_artifacts")
ARTIFACT_DIR.mkdir(exist_ok=True)

joblib.dump(prototype, ARTIFACT_DIR / "tfidf_logreg_pipeline.joblib")
np.save(ARTIFACT_DIR / "train_embeddings.npy", train_embeddings)
retrieval_model[["drugName", "condition", "rating", "satisfaction", "re
    ARTIFACT_DIR / "train_model.parquet"
)
(ARTIFACT_DIR / "embed_model_name.txt").write_text(EMBED_MODEL_NAME)

print("✓ Improved artifacts exported to:", ARTIFACT_DIR.resolve())
for f in sorted(ARTIFACT_DIR.iterdir()):
    print(" –", f.name)
```

```
✓ Improved artifacts exported to: /content/model_artifacts
 – embed_model_name.txt
 – tfidf_logreg_pipeline.joblib
 – train_embeddings.npy
 – train_model.parquet
```

14. Professional Medication Review Intelligence Dashboard

The final interface now behaves like a real review-intelligence product rather than a generic chatbot. Medication, condition, and review are supplied as model context. The classifier is trained on 90,000 reviews, while a separate 35,000-review library keeps semantic retrieval responsive.

```
%pip install -q streamlit plotly
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.5/10.5 MB 32.0 MB/s eta
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 11.4/11.4 MB 121.4 MB/s eta
```

```
%%writefile streamlit_app.py
import html
import re
from pathlib import Path

import joblib
import numpy as np
import pandas as pd
import plotly.graph_objects as go
import streamlit as st
from sentence_transformers import SentenceTransformer
from sklearn.neighbors import NearestNeighbors

ARTIFACT_DIR = Path(__file__).parent / "model_artifacts"
COLORS = {"low": "#EF6A67", "medium": "#E5A733", "high": "#18A48F"}
SOFT = {"low": "#FFF0EF", "medium": "#FFF8E6", "high": "#EAF9F5"}
LABELS = {"low": "Low satisfaction", "medium": "Medium satisfaction",
ICONS = {"low": "↓", "medium": "–", "high": "↑"}
MEDICAL_ADVICE = re.compile(
    r"\b(should i|can i take|dose|dosage|prescribe|stop taking|best d
    flags=re.IGNORECASE,
)
EXPERIENCES = {
    "Nausea": r"\b(nausea|nauseous|sick to my stomach)\b",
    "Dizziness": r"\b(dizzy|dizziness|lightheaded)\b",
    "Headache": r"\b(headache|migraine)\b",
    "Fatigue": r"\b(tired|fatigue|exhausted|sleepy)\b",
    "Pain": r"\b(pain|painful|aching|ache)\b",
    "Sleep change": r"\b(insomnia|sleep|slept|awake)\b",
    "Mood change": r"\b(anxiety|anxious|depressed|depression|mood)\b"
    "Improvement": r"\b(helped|effective|improved|better|worked|relie
    "Worsening": r"\b(worse|worsened|severe|unbearable|terrible)\b",
}

st.set_page_config(page_title="MedReview Insight", page_icon="💊", la
st.markdown("""
<style>
```

```
@import url('https://fonts.googleapis.com/css2?family=DM+Sans:wght@40(
:root{--navy:#091F35;--navy2:#123E5B;--teal:#118B83;--mint:#6FDDD0;--
html,body,[class*="css"]{font-family:'DM Sans',sans-serif;color:var(--
[data-testid="stAppViewContainer"]{background:radial-gradient(circle a
[data-testid="stHeader"]{background:rgba(250,252,253,.82);backdrop-fi
[data-testid="stSidebar"]{background:linear-gradient(180deg,#F8FCFC,#E
.block-container{max-width:1450px;padding-top:1.7rem;padding-bottom:4
.hero{position:relative;overflow:hidden;background:linear-gradient(11
.hero:after{content:"";position:absolute;width:330px;height:330px;rig
.kicker{color:#8CEADF;font-size:.74rem;font-weight:700;letter-spacing
.hero h1{color:white!important;font-size:2.5rem;letter-spacing:-.035em
.hero p{color:#D8EEF0;margin:.2rem 0;max-width:850px}
.chips{display:flex;gap:.45rem;flex-wrap:wrap;margin-top:1rem}
.chip{border:1px solid rgba(255,255,255,.18);background:rgba(255,255,
.notice{background:#FFF9E8;border:1px solid #F0DA9B;border-left:4px so
.hero-subtitle{font-size:1.16rem!important;font-weight:700;color:#D9FA
.hero-support{font-size:.96rem!important;color:#D6E6EC!important;marg
.hero-team{font-size:.78rem!important;color:#AFC8D2!important;margin:
.sidebar-warning{font-size:.76rem;line-height:1.48;color:#667985;backg
.metric-grid{display:grid;grid-template-columns:repeat(4,1fr);gap:.8re
.metric-card{background:white;border:1px solid var(--line);border-radi
.metric-label{font-size:.72rem;color:#6D818C;text-transform:uppercase;
.metric-value{font-size:1.48rem;font-weight:700;color:#113A55;margin-t
.section-card{background:white;border:1px solid var(--line);border-rad
.result-card{border-radius:19px;padding:1.2rem;text-align:center;borde
.result-icon{font-size:2rem;font-weight:700}.result-label{font-size:1
.quality-bar{height:8px;background:#E7EFF1;border-radius:999px;margin
.evidence-pill{display:inline-block;border:1px solid #CDE3E3;backgroun
.experience-pill{display:inline-block;background:#F2F6F8;color:#34546B
.review-card{background:white;border:1px solid var(--line);border-radi
.review-meta{color:#667D89;font-size:.78rem;margin-bottom:.5rem}.sourc
.grounded-answer{background:linear-gradient(135deg,#EDF9F6,#F4FAFC);bo
.empty-state{text-align:center;padding:3.2rem 1rem;color:#6A7F8A}.empt
.sidebar-logo{font-size:1.12rem;font-weight:700;color:#103B56;margin-b
.stButton>button{border-radius:11px!important;border:1px solid #CFE0E4
.stButton>button[kind="primary"]{background:linear-gradient(135deg,#0B
[data-testid="stTextInput"] input,[data-testid="stTextArea"] textarea{
[data-baseweb="tab-list"]{gap:.35rem;background:#E9F1F3;padding:.3rem;
[data-baseweb="tab"]{border-radius:10px;padding:.6rem 1rem}[aria-selec
@media(max-width:900px){.metric-grid{grid-template-columns:repeat(2,1f
</style>
""", unsafe_allow_html=True)

# Presentation-only visual enhancements. CSS animations do not affect
st.markdown("""
```

```
<style>
.hero{border-radius:26px;padding:1.5rem 2.25rem 1.35rem;box-shadow:0 7
.hero h1{font-size:2.55rem;margin:.12rem 0 .3rem}
.hero:before{content:"";position:absolute;width:380px;height:380px;rig
.hero h1,.hero p,.hero .kicker,.hero .chips{position:relative;z-index
.hero-visual{position:absolute;z-index:2;right:1.8rem;top:50%;width:27
.hero-visual .glass{fill:rgba(255,255,255,.09);stroke:rgba(187,255,247
@keyframes heroEntrance{from{opacity:0;transform:translateY(-14px)}to{
@keyframes floatingGlow{0%,100%{transform:translateY(0) scale(1)}50%{t
.workflow-ribbon{position:relative;display:grid;grid-template-columns
.workflow-ribbon:before{content:"";position:absolute;left:-35%;bottom
.workflow-step{position:relative;padding:.95rem 1rem;text-align:center
.workflow-number{display:inline-flex;align-items:center;justify-conter
.metric-card{position:relative;overflow:hidden;background:linear-grad
.metric-card:after{content:"";position:absolute;width:75px;height:75px
.metric-card{animation:metricRise .55s both}.metric-card:nth-child(2){
.section-card{animation:cardEntrance .45s ease-out}@keyframes cardEntr
.result-card{border-radius:22px;box-shadow:0 12px 30px rgba(21,58,75,
.quality-fill{animation:progressAnimation 1s ease-out}@keyframes progr
.evidence-pill,.review-card,.stButton>button{transition:transform .16s
.review-card{border-left:4px solid #168F88}.review-card:hover{transfor
.source-badge{background:linear-gradient(135deg,#123E5B,#168F88)}
.empty-state{border:1px dashed #C9DBDF;border-radius:20px;background:#
.stButton>button:hover{transform:translateY(-2px);box-shadow:0 7px 17p
.stButton>button[kind="primary"]{position:relative;overflow:hidden}.st
@keyframes visualFloat{0%,100%{transform:translateY(-50%)}50%{transfor
@keyframes drawPulse{0%{stroke-dashoffset:240;opacity:.25}35%,72%{stro
@keyframes orbGlow{0%,100%{opacity:.35;transform:scale(.82)}50%{opacit
@keyframes workflowSweep{0%{left:-35%;opacity:0}15%{opacity:1}75%{opac
@keyframes stepBreathe{0%,100%{box-shadow:0 0 0 0 rgba(69,200,212,0)}7
@keyframes metricRise{from{opacity:0;transform:translateY(12px)}to{opa
@keyframes buttonShine{0%,68%{left:-120%}88%,100%{left:150%}}
@media(max-width:900px){.workflow-ribbon{grid-template-columns:repeat
@media(prefers-reduced-motion:reduce){*,*:before,*:after{animation-dur
</style>
""", unsafe_allow_html=True)

def clean_text(value):
    return re.sub(r"\s+", " ", html.unescape(str(value))).strip()

@st.cache_resource(show_spinner="Loading review intelligence models…")
def load_artifacts():
    required = ["tfidf_logreg_pipeline.joblib", "train_embeddings.npy",
    missing = [name for name in required if not (ARTIFACT_DIR / name).
    if missing:
```

```
            raise FileNotFoundError("Missing artifacts: " + ", ".join(miss
        pipeline = joblib.load(ARTIFACT_DIR / required[0])
        embeddings = np.load(ARTIFACT_DIR / required[1])
        corpus = pd.read_parquet(ARTIFACT_DIR / required[2])
        embedder = SentenceTransformer((ARTIFACT_DIR / required[3]).read_t
        retriever = NearestNeighbors(n_neighbors=5, metric="cosine", algo
        return pipeline, embedder, retriever, corpus

    try:
        pipeline, embedder, retriever, corpus = load_artifacts()
    except Exception as exc:
        st.error(str(exc)); st.stop()

    def model_text(review, drug, condition):
        return f"drug_{clean_text(drug).replace(' ','_')} condition_{clea

    def evidence_terms(text, label, top_n=8):
        vectorizer, classifier = pipeline.named_steps["tfidf"], pipeline.
        row = vectorizer.transform([text])
        if row.nnz == 0: return []
        names = np.asarray(vectorizer.get_feature_names_out())
        idx = list(classifier.classes_).index(label)
        contribution = row.data * classifier.coef_[idx, row.indices]
        selected = names[row.indices[np.argsort(contribution)[::-1][:top_
        return [str(term).split("__", 1)[-1] for term in selected]

    def extract_experiences(review):
        text = clean_text(review).lower()
        return [name for name, pattern in EXPERIENCES.items() if re.searc

    def analyze(review, drug, condition, k=3):
        text = model_text(review, drug, condition)
        raw_probs = pipeline.predict_proba([text])[0]
        probabilities = dict(zip(pipeline.classes_, map(float, raw_probs)
        label = max(probabilities, key=probabilities.get)
        query = embedder.encode([text], normalize_embeddings=True)
        distances, indices = retriever.kneighbors(query, n_neighbors=min(
        similar = corpus.iloc[indices[0]].copy()
        similar["similarity"] = np.clip(1 - distances[0], 0, 1)
        return {
            "review": review, "drug": drug, "condition": condition, "labe
            "probabilities": probabilities, "terms": evidence_terms(text,
            "experiences": extract_experiences(review), "similar": simila
            "retrieval_quality": float(similar["similarity"].mean()),
        }
```

```
def probability_chart(probabilities):
    order = ["low", "medium", "high"]
    fig = go.Figure(go.Bar(
        x=[probabilities.get(x, 0) for x in order], y=[LABELS[x] for x
        marker=dict(color=[COLORS[x] for x in order], line=dict(width=
        text=[f"{probabilities.get(x, 0):.1%}" for x in order], textpo
    ))
    fig.update_layout(height=270, margin=dict(l=10, r=40, t=18, b=20),
        xaxis=dict(tickformat=".0%", range=[0, 1.08], title="Model pro
        yaxis=dict(autorange="reversed"), plot_bgcolor="rgba(0,0,0,0)"
    return fig

def distribution_chart(frame, field, title):
    counts = frame[field].value_counts().reindex(["low", "medium", "hi
    fig = go.Figure(go.Pie(labels=[LABELS[x] for x in counts.index], v
        marker=dict(colors=[COLORS[x] for x in counts.index], line=dic
    fig.update_layout(title=dict(text=title, font=dict(size=14)), heig
        paper_bgcolor="rgba(0,0,0,0)", legend=dict(orientation="h", y=
    return fig

def metric_cards(items):
    cards = "".join(f'<div class="metric-card"><div class="metric-labe
    st.markdown(f'<div class="metric-grid">{cards}</div>', unsafe_allo

def render_review_cards(similar):
    for i, (_, row) in enumerate(similar.iterrows(), 1):
        excerpt = clean_text(row["review_clean"])
        if len(excerpt) > 430: excerpt = excerpt[:430].rstrip() + "…"
        st.markdown(
            f'<div class="review-card"><div class="review-meta"><span
            f'<b>{html.escape(str(row["drugName"]))}</b> · {html.escap
            f'Rating {row["rating"]}/10 · Semantic match {row["simila
            f'<div>{html.escape(excerpt)}</div></div>', unsafe_allow_h

def grounded_summary(result):
    label, confidence = result["label"], result["probabilities"][resu
    evidence = ", ".join(result["terms"][:4]) if result["terms"] else
    experiences = ", ".join(result["experiences"][:4]) if result["expe
    return (f"This review is classified as **{LABELS[label].lower()}**
            f"The strongest model signals include **{evidence}**. Dete
            f"The explanation is grounded in {len(result['similar'])}

def answer_followup(question, result):
    q = question.lower(); label = result["label"]; confidence = resul
```

```
        if "why" in q or "classif" in q:
            terms = ", ".join(result["terms"][:6]) or "the review's overa
            return f"The model selected **{LABELS[label].lower()}** at **
        if "side effect" in q or "symptom" in q or "experience" in q:
            found = ", ".join(result["experiences"]) or "No common symptor
            return f"Detected experience themes: **{found}**. These are te
        if "similar" in q or "source" in q or "evidence" in q:
            best = result["similar"].iloc[0]
            return f"The strongest retrieved source is a review for **{bes
        if "confiden" in q or "uncertain" in q:
            spread = sorted(result["probabilities"].values(), reverse=True
            margin = spread[0] - spread[1]
            return f"Confidence is **{confidence:.1%}** and the margin ove
        return grounded_summary(result)

    st.markdown("""<div class="hero"><div class="hero-copy"><div class="k
    st.markdown("""
    <div class="workflow-ribbon">
      <div class="workflow-step"><span class="workflow-number">1</span><sp
      <div class="workflow-step"><span class="workflow-number">2</span><sp
      <div class="workflow-step"><span class="workflow-number">3</span><sp
      <div class="workflow-step"><span class="workflow-number">4</span><sp
    </div>
    """, unsafe_allow_html=True)

    def clear_workspace():
        # Widget keys must be reset explicitly so Streamlit does not resto
        # previous browser values on the following rerun.
        st.session_state["drug"] = ""
        st.session_state["condition"] = ""
        st.session_state["review_input"] = ""
        st.session_state.pop("analysis", None)
        st.session_state.pop("followups", None)

    with st.sidebar:
        st.markdown('<div class="sidebar-logo">MedReview Workspace</div><
        drug = st.text_input("Drug name", key="drug", placeholder="e.g., I
        condition = st.text_input("Condition", key="condition", placeholde
        k = st.slider("Evidence sources", 2, 5, 3)
        st.markdown("---")
        st.caption("Prediction: hybrid word/character TF-IDF. Retrieval: M
        st.button("Clear workspace", use_container_width=True, on_click=cl
        st.markdown('<div class="sidebar-warning">⚕ <b>Research and educ

    metric_cards([
```

```
        ("Model training reviews", "90,000"),
        ("Semantic search library", f"{len(corpus):,}"),
        ("Medications represented", f"{corpus['drugName'].nunique():,}"),
        ("Average rating", f"{corpus['rating'].mean():.1f} / 10"),
    ])

    tab_analysis, tab_explore, tab_method = st.tabs(["Review Analysis", "I

    with tab_analysis:
        def use_example(sample_drug, sample_condition, sample_text):
            st.session_state.drug = sample_drug
            st.session_state.condition = sample_condition
            st.session_state.review_input = sample_text

        left, right = st.columns([1.45, .75])
        with left:
            review = st.text_area(
                "Patient medication review",
                height=155,
                key="review_input",
                placeholder="Describe the medication experience, including
            )
        with right:
            st.markdown("**Try an example**")
            examples = [
                ("Side effects", "Lisinopril", "High Blood Pressure", "It
                ("Strong benefit", "Sertraline", "Depression", "This worke
                ("Mixed outcome", "Gabapentin", "Neuropathic Pain", "I not
            ]
            for i, (example_label, sample_drug, sample_condition, sample_
                st.button(
                    example_label,
                    key=f"example_{i}",
                    use_container_width=True,
                    on_click=use_example,
                    args=(sample_drug, sample_condition, sample_text),
                )
        if st.button("Analyze review", type="primary", use_container_widtl
            if not clean_text(review): st.warning("Enter a review before
            elif MEDICAL_ADVICE.search(review): st.warning("Please descril
            else:
                with st.spinner("Analyzing satisfaction signals and retri
                    st.session_state.analysis = analyze(review, drug or "

        result = st.session_state.get("analysis")
```

```
        if not result:
            st.markdown('<div class="empty-state"><div class="empty-icon">
        else:
            label, confidence = result["label"], result["probabilities"][
            metric_cards([
                ("Prediction", LABELS[label]), ("Confidence", f"{confiden
                ("Evidence quality", f"{result['retrieval_quality']:.0%}"
            ])
            c1, c2 = st.columns([.75, 1.5])
            with c1:
                st.markdown(f'<div class="result-card" style="background:
            with c2: st.plotly_chart(probability_chart(result["probabilit
            st.markdown('<div class="section-card"><b>Grounded interpreta
            ev1, ev2 = st.columns(2)
            with ev1:
                st.markdown("#### Language evidence")
                pills = "".join(f'<span class="evidence-pill">{html.escape
                st.markdown(pills or "No high-weight evidence term identi
            with ev2:
                st.markdown("#### Experience themes")
                pills = "".join(f'<span class="experience-pill">{html.esca
                st.markdown(pills or "No common experience category explic
            st.markdown("#### Similar patient-review evidence")
            st.caption("Retrieved sources are patient anecdotes and are no
            render_review_cards(result["similar"])
            st.markdown("#### Ask about this result")
            questions = ["Why this classification?", "Which experiences wo
            qcols = st.columns(4)
            for i, question in enumerate(questions):
                if qcols[i].button(question, key=f"follow_{i}", use_conta
                    st.session_state.followups = [(question, answer_follow
            for question, answer in st.session_state.get("followups", [])
                st.markdown(f'<div class="grounded-answer"><b>{html.escape

    with tab_explore:
        filtered = corpus.copy()
        if drug and drug.lower() != "unknown drug": filtered = filtered[f
        if condition and condition.lower() != "unknown condition": filtere
        st.caption("This tab summarizes the 35,000-review semantic search
        if filtered.empty:
            st.info("No matching review records were found. Try a broader
        else:
            metric_cards([("Searchable reviews", f"{len(filtered):,}"), ("
            p1, p2 = st.columns(2)
            with p1: st.plotly_chart(distribution_chart(filtered, "satisfa
```

```
            with p2:
                rating_counts = filtered["rating"].value_counts().sort_in
                fig = go.Figure(go.Bar(x=rating_counts.index, y=rating_co
                fig.update_layout(title="Rating distribution", height=300
                st.plotly_chart(fig, use_container_width=True)
            st.markdown("#### Most represented medications")
            top = filtered["drugName"].value_counts().head(10).rename_axi
            st.dataframe(top, use_container_width=True, hide_index=True)

    with tab_method:
        st.markdown("""
        ### A grounded research workflow

        1. **Contextualize:** combine medication name, condition, and pat
        2. **Predict:** classify satisfaction with word- and character-le
        3. **Explain:** identify the highest-contributing text features f
        4. **Retrieve:** use MiniLM embeddings to search 35,000 semantica
        5. **Ground:** show source cards and provide deterministic follow-

        **Technology:** hybrid word- and character-level TF-IDF, balanced

        This design avoids a large generative model, so the interface rema
        """)
```

```
Writing streamlit_app.py
```

14.1 Launch the app

Run the next cell after artifact export. It starts Streamlit and displays the professional app directly inside Colab. No ngrok token is required.

```
import subprocess, sys, time
import requests
from IPython.display import HTML, display
from google.colab.output import eval_js

try:
    streamlit_proc.terminate()
except Exception:
    pass
```

```
log_path = "/content/streamlit_logs.txt"
streamlit_proc = subprocess.Popen(
    [sys.executable, "-m", "streamlit", "run", "streamlit_app.py",
     "--server.port", "8501", "--server.address", "0.0.0.0",
     "--server.headless", "true", "--server.enableCORS", "false",
     "--server.enableXsrfProtection", "false", "--browser.gatherUsage
    stdout=open(log_path, "w"), stderr=subprocess.STDOUT,
)
healthy = False
for _ in range(60):
    if streamlit_proc.poll() is not None:
        break
    try:
        response = requests.get("http://127.0.0.1:8501/_stcore/health'
        if response.ok:
            healthy = True
            break
    except requests.RequestException:
        pass
    time.sleep(1)

if not healthy:
    print(Path(log_path).read_text()[-6000:])
    raise RuntimeError("Streamlit did not become healthy. The actual 

app_url = eval_js("google.colab.kernel.proxyPort(8501)")
print("✓ Streamlit is healthy. Click the button below (do not use an U
display(HTML(
    f'<a href="{app_url}" target="_blank" '
    'style="display:inline-block;background:#0E7C75;color:white;paddi
    'border-radius:10px;text-decoration:none;font-weight:700;font-siz
    'Open MedReview Insight ↗</a>'
))
print("Direct link:", app_url)
```

```
✓ Streamlit is healthy. Click the button below (do not use an Untitled
```

Open MedReview Insight ↗

Direct link: https://8501-gpu-t4-s-kkb-ass1c1-a3xwbbggxmbz-c.asia-south

```
# Optional: inspect recent Streamlit logs if the app does not appear.
print(Path("/content/streamlit_logs.txt").read_text()[-4000:])
```

```
2026-08-16 09:18:55.332 Uvicorn server started on 0.0.0.0:8501

  You can now view your Streamlit app in your browser.

  Local URL: http://localhost:8501
  Network URL: http://172.28.0.12:8501
  External URL: http://34.177.93.220:8501
```

```
# Optional: stop the app.
# streamlit_proc.terminate()
```

```
# Security note: no ngrok token is stored in this notebook.
# The Colab port proxy above is the supported launch path for this pr
```

```
print("Notebook complete. Use Section 14.1 to open the app.")
```

```
Notebook complete. Use Section 14.1 to open the app.
```

Run notes

- Use a T4 GPU for faster MiniLM encoding.
- Upload the two Drugs.com CSV files; names with or without `(1)` are accepted.
- Run cells in order. The app launches only after the improved model and retrieval artifacts are exported.
- Predictive performance is reported on the untouched official test split sample using both accuracy and macro-F1.